

Rocktivate

Project Documentation

Generated 2026-08-11

Rocktivate Documentation

- [FEATURES.md](#) — what's implemented, screen by screen, with status
- [ARCHITECTURE.md](#) — tech stack, folder structure, navigation, data flow
- [DATA_MODEL.md](#) — Supabase tables/views and external data sources
- [KNOWN_ISSUES.md](#) — open issues (incl. a live security issue) and what's already been fixed

These are living documents generated from the current state of the code (last full pass: 2026-08-11). Update them as features change — they reflect what's actually implemented, not a roadmap.

Rocktivate — Feature Tracker

Last updated: 2026-08-11

Status legend:  Implemented ·  Partial / needs wiring ·  Not started

This tracks what exists in the codebase today (`lib/`), so it can be updated as work continues. Update the status and "Notes" column as features change — this file reflects code, not intentions.

Auth & Onboarding

Feature	Status	Files	Notes
Email/password sign in		<code>sign_up_signin/login/</code>	
Email/password sign up		<code>sign_up_signin/create/</code>	
Forgot password		<code>sign_up_signin/forgot_password/</code>	
Google sign-in		<code>auth/auth_manager.dart</code> (<code>GoogleSignInManager</code> mixin)	Mixin declared, not applied to <code>SupabaseAuthManager</code> . No button on login screen.
Apple sign-in		<code>auth/auth_manager.dart</code> (<code>AppleSignInManager</code> mixin), <code>sign_in_with_apple</code> dependency	Package installed, mixin declared, but not wired into <code>SupabaseAuthManager</code> or the login UI.
Facebook / Microsoft / Github sign-in		<code>auth/auth_manager.dart</code>	Mixins declared only, no implementation, no UI.
Anonymous sign-in		<code>auth/auth_manager.dart</code> (<code>AnonymousSignInManager</code> mixin)	Declared, unused.
Onboarding walkthrough		<code>onboarding/onboarding/</code> , <code>onboarding/onboard/</code>	
Post-login routing ("director")		<code>login_director/</code>	Routes user based on auth/onboarding state.

Community

Feature	Status	Files	Notes
Browse communities		<code>pages/community/</code>	
Community detail page		<code>pages/community_detail/</code>	
Community members list		<code>pages/community_members/</code>	

Feature	Status	Files	Notes
Community settings (manage/edit)	✓	pages/community_settings/	
My Communities list	✓	pages/my_communities/	
Join requests	✓	backed by community_requests table	
Private communities	✓	communities.isPrivate field exists	Confirm UI enforces it.

Social Feed

Feature	Status	Files	Notes
Public feed	✓	pages/public_feed/	
Create post (text)	✓	components/new_group_post_widget	
Post with images	✓	components/post_images_widget	
Comments	✓	backed by comments table	
Likes (posts & comments)	✓	backed by likes table	
Group posts	✓	posts.isGroupPost , posts.groupLink	Links a post to a community.

Needs (Community Help Requests)

Feature	Status	Files	Notes
Browse needs	✓	pages/needs/	
Need detail page	✓	need_details/	
Create a need	✓	components/create_new_need_widget	
Mark need fulfilled	✓	needs.fulfilled field exists	Confirm UI has a toggle/action.

Free Stuff (Giveaways)

Feature	Status	Files	Notes
Browse free items	✓	pages/free_stuff/	
Item detail page	✓	pages/freebie_details/	
My Freebies (manage own listings)	✓	pages/my_freebies/	
Freebie settings (create/edit)	✓	pages/freebie_settings/	
Availability toggle	✓	freebies.isAvailable field exists	

Accommodation

Feature	Status	Files	Notes
Property listings	✓	pages/accomodation/ , backed by property table	Rich schema: type, address, price, bedrooms/bathrooms, parking, status.
Accommodation detail page	✓	accomodation_details/	

Chat / Messaging

Feature	Status	Files	Notes
Conversation list	✓	chat/my_messages/ , conversation_list view	
1:1 chat thread	✓	chat/chat/ , chat/message_bubble/	
Start new chat	✓	chat/new_chat/	
Read receipts	●	conversation_participants.lastReadAt field exists	Confirm UI reflects read state.

Bible Reader

Feature	Status	Files	Notes
List Bible translations	✓	bible/bibles/	Source: api.bible (rest.api.bible/v1/).
List books	✓	bible/books/	
List chapters	✓	bible/chapters/	
Chapter reading view	✓	bible/chapter_data/	

AI Bible Assistant

Feature	Status	Files	Notes
Ask-AI Q&A	✓	components/a_i_response_widget , backend/api_requests/api_calls.dart (ChatGPTCall)	Calls OpenAI gpt-4o-mini directly from the client with a system prompt framing it as a "bible scholar." Markdown-rendered response.
Conversation history for AI chat	■	—	Each question appears to be a single-shot call, no thread persistence found. Verify.

Profile

Feature	Status	Files	Notes
View/edit profile	✓	<code>profile/</code> , backed by <code>people</code> table	Fields: name, birthday, email, country, postcode, gender, bio, profile image.
Onboarding completion flag	✓	<code>people.hasCompletedOnboarding</code>	

Search

Feature	Status	Files	Notes
Global search	✓	<code>search/</code>	Verify which entities it covers (posts/needs/freebies/people/communities).

Home

Feature	Status	Files	Notes
Home hub screen	✓	<code>home/</code>	Embeds AI assistant and a webview (<code>webviewx_plus</code>) — confirm what the webview loads.
Dynamic menu	🟡	<code>menu_items</code> table (title, image, gradient, <code>orderId</code> , enabled)	Table exists for a configurable menu; confirm it's rendered somewhere (not yet traced to a widget in this pass).

Platform / Infra

Item	Status	Notes
Supabase backend (Postgres + Auth + Realtime + Storage)	✓	
Routing	✓	<code>go_router</code> via <code>flutter_flow/nav/nav.dart</code>
State management	✓	<code>provider</code>
Deep linking	✓	<code>app_links</code> package present
Localization scaffolding	✓	<code>flutter_localizations</code> dependency present — verify actual translated strings exist
Web build	✓	Confirmed working (<code>flutter run -d chrome</code>)
Android build	✓ (assumed)	Not explicitly re-verified this session
iOS build	✓	Confirmed working on simulator as of 2026-08-11, after installing CocoaPods
macOS build	🟡	Should work now that CocoaPods is installed — not explicitly re-verified

Open Questions to Resolve

- Which entities does the search screen actually query?
- Does the `menu_items` table drive a real UI, or is it unused/legacy?
- Is the AI assistant single-turn only, or does it maintain conversation context?
- Are Google/Apple sign-in intentionally disabled, or mid-migration?

Rocktivate — Architecture Overview

Last updated: 2026-08-11

Origin

This app was scaffolded/exported by **FlutterFlow** and is now maintained as a hand-editable Flutter codebase. That explains the file-naming convention (`*_widget.dart` + `*_model.dart` pairs per screen) and the volume of boilerplate lint warnings (unused imports, missing `const`) — normal for FlutterFlow exports, not a sign of hand-written sloppiness.

Tech Stack

Layer	Technology
Framework	Flutter 3.44.9 / Dart 3.12.2
Backend	Supabase (Postgres, Auth, Realtime, Storage)
Navigation	<code>go_router</code> , wrapped by FlutterFlow's <code>flutter_flow/nav/nav.dart</code>
State management	<code>provider</code>
AI	OpenAI <code>gpt-4o-mini</code> via direct REST call (no server-side proxy)
Bible content	<code>api.bible</code> public REST API
Deep linking	<code>app_links</code>
Local storage	<code>hive</code> , <code>sqflite</code> , <code>shared_preferences</code>
Auth	<code>supabase_flutter</code> (email/password wired; social providers scaffolded but unused)

Project Structure

```
lib/
├─ auth/                Auth abstraction (AuthManager) + Supabase implementation
├─ backend/
│   └─ api_requests/    Raw REST calls (OpenAI, api.Bible) – api_calls.dart
│       └─ supabase/
│           └─ database/tables/  One Dart model per Supabase table/view
│               └─ storage/      Supabase Storage helpers
├─ components/          Shared, reusable widgets used across screens
├─ flutter_flow/        FlutterFlow runtime support code (theme, nav, utils, custom wic
├─ pages/               Most feature screens (community, feed, needs, freebies, accomm
├─ home/, profile/, search/, chat/, bible/, onboarding/,
└─ sign_up_signin/, login_director/, need_details/
```



```
| accomodation_details/  Additional top-level screens
└─ main.dart             App entry point
```

Each screen typically has: - `*_widget.dart` — the UI (StatefulWidget wrapping a Model) - `*_model.dart` — form controllers, animation controllers, disposal logic

Navigation

All routes are registered in `lib/flutter_flow/router.dart` / `lib/flutter_flow/nav/nav.dart` using `go_router`. Route names/paths are static members on each screen's widget class (`SomeWidget.routeName`, `SomeWidget.routePath`). To find every screen in the app, grep that file rather than walking directories — some widgets (e.g. dialogs, bottom sheets) live outside `pages/` and don't have routes.

Data Flow

- Supabase tables are wrapped in typed Dart classes under `backend/supabase/database/tables/`, generated from the Supabase schema. Several **views** exist alongside base tables to pre-join data for the UI (e.g. `postsview`, `messages_with_sender`, `community_members_with_full_name`) — prefer reading from these views over re-joining client-side.
- The OpenAI and api.Bible calls in `backend/api_requests/api_calls.dart` are called directly from the client with the API key/config inline — there is no backend proxy for these two integrations. See `docs/DATA_MODEL.md` for the full table list.

Known Environment Setup Requirements

- **CocoaPods** is required for iOS/macOS builds (native plugin linking for `sqflite`, `app_links`, `sign_in_with_apple`, `image_picker`, `webview_flutter`, `video_player`, etc.). macOS's bundled system Ruby (2.6.10) is too old for modern CocoaPods — install via Homebrew (`brew install cocoapods`), not `sudo gem install cocoapods`.
- Supabase URL and anon key are hardcoded in `lib/backend/supabase/supabase.dart` (this is normal for the anon key, which is designed to be public — RLS policies enforce real security server-side).
- ⚠️ **Security issue, not just a TODO:** `ChatGPTCall` in `backend/api_requests/api_calls.dart` has a **live OpenAI secret key hardcoded in plaintext** (`Authorization: Bearer sk-proj-...`) and calls the OpenAI API directly from the client. This key ships inside every build of the app and can be extracted by anyone who downloads it. See `docs/KNOWN_ISSUES.md` for the recommended fix (rotate the key, move the call behind a Supabase Edge Function).

See `docs/KNOWN_ISSUES.md` for build issues encountered and fixed, and `docs/FEATURES.md` for what's implemented.

Rocktivate — Data Model (Supabase)

Last updated: 2026-08-11

Generated from `lib/backend/supabase/database/tables/*.dart`. This reflects the Dart model layer, not a live schema dump — re-verify against Supabase directly before relying on it for migrations.

Base Tables

Table	Key fields	Used by
people	uuid, firstName, lastName, preferredName, birthday, email, userUUID, profileImage, country, postCode, gender, hasCompletedOnboarding, bio	Profile, onboarding, auth linkage
communities	uuid, createdAt, createdBy, title, tileImage, headerImage, isPrivate	Community pages
community_members	uuid, createdAt, peopleLink, communityLink	Community membership
community_requests	uuid, createdAt, peopleLink, communityLink	Join requests
posts	uuid, description, createdBy, createdAt, isGroupPost, groupLink	Public feed, community feed
comments	uuid, createdAt, createdBy, text, postLink	Feed
likes	uuid, postLink, commentLink, personLink	Feed reactions
needs	uuid, createdAt, createdBy, title, description, fulfilled	Needs feature
freebies	uuid, createdAt, createdBy, title, description, isAvailable	Free Stuff feature
property	id, title, description, propertyType, address, city, state, postcode, country, price, currency, bedrooms, bathrooms, parkingSpaces, mainImageUrl, status, createdBy, createdAt, updatedAt	Accommodation feature
conversations	id, createdAt	Chat
conversation_participants	id, conversationId, userId, joinedAt, lastReadAt	Chat
messages	uuid, createdAt, body, sentBy, sentTo	Legacy/simple messaging? (see <code>chat_messages</code> below)
chat_messages	id, conversationId, senderId, body, createdAt	Chat thread messages
countries	uuid, name, countryID	Profile country picker, address forms
menu_items	uuid, orderIndex, title, image, gradient (x2), gradientDegree, enabled, subtitle	Possibly a configurable home-screen menu — verify it's

Table	Key fields	Used by
		actually rendered (see FEATURES.md open questions)

Views (pre-joined, read-optimized)

View	Key fields	Purpose
<code>postsvview</code>	uuid, description, createdBy, createdAt, isGroupPost, groupLink, fullname, commentscount, reactioncount, profileImage	Feed row with author + counts pre-joined — use this instead of joining <code>posts</code> + <code>people</code> + <code>comments</code> + <code>likes</code> client-side
<code>community_members_with_full_name</code>	communityMemberUUID, createdAt, peopleLink, communityLink, fullName, email, profileImage	Member list rendering
<code>community_requests_with_full_name</code>	communityMemberUUID, createdAt, peopleLink, communityLink, fullName, email, profileImage	Join-request review UI
<code>conversation_list</code>	conversationId, viewerId, otherUserId, otherUserName, otherUserImage, lastMessage, lastMessageAt	Chat inbox list
<code>conversation_messages</code>	messageUUID, sentAt, messageBody, sentBy, senderFullName, sentTo, recipientFullName	Chat thread with names pre-joined
<code>messages_with_sender</code>	messageUUID, sentAt, messageBody, senderUUID, senderFullName, senderProfileImage, recipientUUID, recipientFullName, recipientProfileImage	Same pattern for <code>messages</code> table

Notable Relationships (inferred from field names)

- `community_members.peopleLink` → `people`, `community_members.communityLink` → `communities`
- `posts.groupLink` → `communities` (when `isGroupPost` is true)
- `comments.postLink` → `posts`
- `likes.postLink` → `posts`, `likes.commentLink` → `comments`, `likes.personLink` → `people`
- `chat_messages.conversationId` → `conversations`, `chat_messages.senderId` → `people /auth user`
- `property.createdBy`, `needs.createdBy`, `freebies.createdBy`, `posts.createdBy`, `communities.createdBy` → likely all reference `people` or the Supabase auth user id

External Data Sources (non-Supabase)

Source	Used for	Notes
<code>api.bible</code> (<code>rest.api.bible/v1/</code>)	Bible translations, books, chapters, chapter text	<code>backend/api_requests/api_calls.dart</code> : <code>BiblesCall</code> , <code>BooksCall</code> , <code>ChapterCall</code> , <code>ChapterDataCall</code>
OpenAI (<code>api.openai.com/v1/chat/completions</code> , model <code>gpt-4o-mini</code>)	AI Bible-assistant Q&A	<code>ChatGPTRCall</code> — see <code>docs/KNOWN_ISSUES.md</code> : API key is currently hardcoded in the client

To verify against live schema

This doc was built by reading generated Dart model files, which only expose fields FlutterFlow's schema generator picked up. Double-check the actual Supabase schema (Table Editor or `supabase db dump`) for: - Foreign key constraints (the relationships above are inferred, not confirmed) - RLS (Row Level Security) policies — especially important since the client talks to Supabase directly - Any columns not surfaced in the Dart models

Rocktivate — Known Issues

Last updated: 2026-08-11

Status legend: ● Open · ● In progress · ✓ Resolved

● Open

1. OpenAI API key hardcoded in client source — SECURITY

- **Where:** `lib/backend/api_requests/api_calls.dart`, `ChatGPTCall`, `Authorization: Bearer sk-proj-...`
- **Risk:** The key ships inside every compiled build (web, iOS, Android). Anyone can extract it from the app bundle/JS and use it to make OpenAI calls billed to your account.
- **Fix:**
 - Rotate/revoke the exposed key in the OpenAI dashboard immediately.
 - Move the OpenAI call server-side — e.g. a Supabase Edge Function that holds the key as a secret and the Flutter app calls instead of `api.openai.com` directly.
 - Add rate limiting / auth checks on that Edge Function so only logged-in app users can trigger it.
- **Severity:** High — treat as a live incident, not a backlog item.

2. Social sign-in (Google/Apple/Facebook/Microsoft/Github) declared but not implemented

- **Where:** `lib/auth/auth_manager.dart` — mixins exist (`GoogleSignInManager`, `AppleSignInManager`, etc.) but `SupabaseAuthManager` only mixes in `EmailSignInManager`.
- **Impact:** `sign_in_with_apple` is a dependency but unused; no social buttons on the login screen. Either finish wiring these or remove the unused mixins/dependency to reduce confusion and app size.

3. Analyzer lint debt (1970 issues, all non-blocking)

- **Breakdown:** 1416 info (mostly `prefer_const_constructors`, a few `sized_box_for_whitespace`), 554 warnings (mostly `unused_import`, 58 `unnecessary_non_null_assertion`).
- **Impact:** None on runtime — purely style/perf lint noise typical of FlutterFlow exports. Not urgent, but worth a cleanup pass (`dart fix --apply` handles a large share of these automatically).
- **Two warnings worth a manual look:**
 - `lib/auth/supabase_auth/supabase_auth_manager.dart:59` — `updatePassword` is marked `@override` but doesn't actually override anything in the base `AuthManager` class (not declared there). Harmless but misleading annotation — remove `@override` or add the method to the base class.
 - `lib/flutter_flow/nav/serialization_util.dart:165` — unreachable `default` clause in a switch, already covered by preceding cases.

4. `menu_items` table — unclear if actually used

- Table exists with a full schema (title, image, gradient, order, enabled flag) suggesting a configurable home-screen menu, but no widget was traced back to it in this pass. Confirm whether it's live, legacy, or planned.

✓ Resolved (this session, 2026-08-11)

5. App failed to compile — `font_awesome_flutter: 10.7.0` incompatible with current Flutter SDK

- **Symptom:** `flutter run` failed with `The class 'IconData' can't be extended outside of its library because it's a final class.`
- **Cause:** Flutter 3.44.9 made `IconData` a `final` class; `font_awesome_flutter 10.7.0` predates that change and subclasses it.
- **Fix:** Upgraded to `font_awesome_flutter: 11.0.0` in `pubspec.yaml`. The new version wraps icons in a `FaIconData` type instead of subclassing `IconData`, which required updating 3 call sites that extracted a raw `IconData` from an existing `FaIcon` and re-wrapped it:
- `lib/flutter_flow/flutter_flow_choice_chips.dart`
- `lib/flutter_flow/flutter_flow_icon_button.dart`
- `lib/flutter_flow/flutter_flow_widgets.dart`

Fix pattern: wrap the `IconData` in `FaIconData(...)` before passing to `FaIcon(...)`.

6. App failed to compile — `page_transition: 2.1.0` incompatible with current Flutter SDK

- **Symptom:** `Couldn't find constructor 'CupertinoPageTransitionsBuilder'.`
- **Cause:** Constructor signature changed upstream in Flutter; old package version called it with the old signature.
- **Fix:** Upgraded to `page_transition: 2.2.2`.

7. iOS/macOS builds failed — CocoaPods not installed

- **Symptom:** `flutter run` on the iOS simulator failed with `CocoaPods not installed or not in valid state.`
- **Cause:** CocoaPods is required to link native code for plugins with iOS-side implementations (`sqflite`, `app_links`, `sign_in_with_apple`, `image_picker`, `webview_flutter`, `video_player`, etc.). It wasn't installed on the dev machine.
- **First attempt failed:** `sudo gem install cocoapods` using macOS's bundled system Ruby (2.6.10) — current CocoaPods requires Ruby ≥ 3.0 , and Apple has not updated system Ruby in years.
- **Working fix:** Installed Homebrew, then `brew install cocoapods` (which brings a compatible Ruby). Confirmed working — `pod install` and the Xcode build both completed, app launched on iPhone 17 simulator.
- **Follow-up note:** New terminal shells need `/opt/homebrew/bin` on `PATH` (added to `~/.zprofile` by the Homebrew installer) for `pod` to resolve.

Verified Working (2026-08-11)

- `flutter analyze` — 0 errors (only lint info/warnings, see #3 above)
- `flutter run -d chrome` — builds and launches, Dart VM Service comes up
- `flutter run` on iOS simulator (iPhone 17) — `pod install` + Xcode build succeed, app launches